

Solidify-eml

Verificación Formal Acelerada de Smart Contracts mediante Operadores Universales y Demostración por Intuición Artificial

Convocatoria de Premios de la Cátedra de Ciberseguridad UMA-VirusTotal / Google

Alejandro Moyano Crespillo

Escuela Técnica Superior de Ingeniería de Telecomunicación, Universidad de Málaga, 29071 Málaga, España

Grado en Ingeniería en Sistemas Electrónicos de Telecomunicación

E-mail: mixalejandro04@gmail.com

Julio de 2026

Resumen

La auditoría de seguridad de contratos inteligentes (*Smart Contracts*) es un pilar crítico en la seguridad de la tecnología blockchain, donde los fallos de código resultan en pérdidas financieras inmediatas. Sin embargo, los métodos tradicionales de verificación formal sufren de una alta complejidad y explosión de estados lógicos. Esta propuesta presenta **Solidify-eml**, un framework de auditoría automatizada que acelera la verificación formal de código Solidity. La innovación radica en la convergencia de dos conceptos teóricos: en primer lugar, la simplificación del espacio lógico de la aritmética del programa reduciendo sus invariantes financieras a grafos binarios basados en el *Operador Binario Universal* ($e^x - \ln(y)$); y en segundo lugar, la generación rápida de especificaciones lógicas utilizando modelos de lenguaje masivos para guiar al asistente de pruebas formales **Lean 4** (*Draft, Sketch, and Prove / Vibe Proving*), implementando técnicas de inyección de comentarios en línea para evitar la degradación de éxito del modelo. Durante la estancia de dos meses en GSEC Málaga, se desarrollará un Producto Mínimo Viable (MVP) enfocado en la certificación matemática y automática de la inmunidad frente a ataques de reentrada (*Reentrancy*) en Smart Contracts, acotado a un subconjunto específico de operaciones del lenguaje para garantizar la viabilidad técnica del proyecto.

Índice

1. Resumen del Proyecto	3
2. Definición del Problema	3
2.1. Limitaciones de las Soluciones Actuales	4
3. Solución Propuesta: Solidify-eml	4
3.1. Arquitectura del Sistema	4
3.2. Estado de Desarrollo Actual	5
4. Fundamentos Tecnológicos y Científicos	6
4.1. El Operador Binario Universal de Odrzywolek	6
4.1.1. Aplicación a la Reducción del Espacio de Estados	6
4.1.2. Mitigación del Dominio y Singularidad de Lean 4	6
4.2. Draft, Sketch, and Prove (Vibe Proving)	7
5. Capacidades y Experiencia del Solicitante	7
6. Plan de Trabajo y Cronograma	8
6.1. Fase 1 (Semanas 1-2): Compilador y Parser de Operador Universal	8
6.2. Fase 2 (Semanas 3-4): Pipeline de Intuición Artificial (Vibe Proving)	8
6.3. Fase 3 (Semanas 5-6): Integración del Verificador Formal Lean 4	8
6.4. Fase 4 (Semanas 7-8): Evaluación de Resultados, Optimización y Demostración . . .	9
6.5. Cronograma Resumido	9
7. Recursos Necesarios y Resultado Mínimo Esperado	9
7.1. Recursos de Computación e Infraestructura	9
7.2. Resultado Mínimo Esperado (MVP)	10
7.3. Escalabilidad y Líneas de Trabajo Futuro	10

1. Resumen del Proyecto

El proyecto **Solidify-eml** propone una metodología innovadora para automatizar y acelerar la verificación formal de seguridad en contratos inteligentes (*Smart Contracts*) desarrollados en Solidity. La meta del proyecto es proveer una herramienta práctica que certifique mecánicamente la ausencia de vulnerabilidades críticas —comenzando por los ataques de reentrada (*Reentrancy*)— eliminando los falsos positivos y la necesidad de una costosa intervención humana experta. La prueba de concepto preliminar del framework y los avances del desarrollo se documentarán y compartirán a través de un repositorio público en GitHub (<https://github.com/AleSMC/Solidify-eml>) para facilitar la revisión del tribunal.

Para lograr este objetivo en un marco de desarrollo de dos meses (Julio y Agosto de 2026) compatible con las metas del Producto Mínimo Viable (MVP), la arquitectura de la solución se fundamenta en tres fases tecnológicas:

1. **Traducción y Homogeneización Lógica:** El código fuente Solidity y sus especificaciones lógicas de comportamiento se compilan a un formato intermedio unificado. En lugar de procesar decenas de instrucciones complejas de la EVM, la lógica de las invariantes lógicas y aritméticas financieras (balances de cuenta, transferencias y control de desbordamientos aritméticos) se modela como un árbol lógico estructurado a partir del *Operador Binario Universal* ($e^x - \ln(y)$) desarrollado por el físico Andrzej Odrzywolek. Esto reduce la explosión de estados de verificación, mientras que el flujo de control se gestiona de forma aislada.
2. **Generación Intuitiva de Bocetos de Prueba (Vibe Proving):** Siguiendo el paradigma *Draft, Sketch, and Prove* (DSP) propuesto por Google DeepMind, se utiliza un modelo de lenguaje de gran tamaño (LLM) a través de APIs de computación de Google para actuar como un “intuicionista artificial”. El modelo intuye el parche de seguridad óptimo y genera un borrador informal de la prueba de seguridad del contrato, incluyendo comentarios en línea para guiar de manera óptima al demostrador.
3. **Verificación Mecánica Rígida:** El borrador de la prueba se traduce a un código formal verificable en el asistente de pruebas matemáticas **Lean 4**. Si el código Lean 4 compila con éxito, la corrección del parche y la seguridad del contrato inteligente frente a reentrada quedan matemáticamente demostradas en las propiedades especificadas, proporcionando garantías lógicas fiables.

El desarrollo se llevará a cabo en las instalaciones del centro de ciberseguridad de Google (**GSEC Málaga**), bajo el marco de la Cátedra de Ciberseguridad de la Universidad de Málaga (UMA) en colaboración con VirusTotal.

2. Definición del Problema

El ecosistema blockchain y las finanzas descentralizadas (DeFi) gestionan miles de millones de dólares en activos digitales a través de contratos inteligentes. Debido a la naturaleza inmutable de la tecnología blockchain, cualquier error lógico o vulnerabilidad en el código expuesto a la red puede ser explotado de manera inmediata e irreversible. Según informes de firmas de ciberseguridad, las pérdidas económicas debidas a hackeos de contratos inteligentes ascienden a miles de millones de dólares anuales.

Entre las distintas clases de bugs lógicos, los **ataques de reentrada (Reentrancy)** siguen siendo una de las vulnerabilidades más destructivas. Este fallo ocurre cuando una función de un contrato inteligente realiza una llamada externa a un contrato no confiable antes de actualizar su estado interno (por ejemplo, el saldo de un usuario). El contrato atacante aprovecha esta interacción para volver a invocar la función de forma recursiva antes de que la primera llamada finalice, permitiendo retirar fondos de forma ilimitada.

2.1. Limitaciones de las Soluciones Actuales

Actualmente, las herramientas disponibles para asegurar Smart Contracts se dividen en tres categorías principales, todas con limitaciones críticas:

1. **Analizadores Estáticos y Linters (ej. Slither):** Son herramientas rápidas pero limitadas a buscar patrones sintácticos de vulnerabilidades conocidas. Generan un alto volumen de falsos positivos y son incapaces de detectar fallos lógicos complejos y novedosos.
2. **Fuzzing y Ejecución Simbólica (ej. Echidna, Mythril):** Exploran aleatoriamente o simbólicamente el espacio de entradas para encontrar estados de fallo. Aunque son útiles, no garantizan de forma matemática la ausencia total de fallos lógicos, ya que no cubren el 100 % de las rutas de ejecución lógicas en sistemas complejos.
3. **Verificación Formal Deductiva Clásica (ej. Certora Prover, asistentes manuales):** Es la única metodología capaz de certificar matemáticamente la seguridad del software. Sin embargo, sufre del problema de la **explosión de estados lógicos**. Traducir cada instrucción de Solidity y cada posible cambio de memoria en la máquina virtual genera sistemas de ecuaciones matemáticas tan densos que las herramientas clásicas de resolución (como los resolutores SMT) se saturan, requiriendo que expertos del sector pasen semanas escribiendo manualmente precondiciones y teoremas lógicos extremadamente complejos.

Esta barrera de entrada impide que la verificación formal se adopte de forma generalizada en la industria de desarrollo de software crítico, dejando una ventana de vulnerabilidad expuesta a exploits millonarios.

3. Solución Propuesta: Solidify-eml

Para solventar las limitaciones del análisis estático y la complejidad extrema de la verificación formal deductiva clásica, **Solidify-eml** introduce una nueva arquitectura de auditoría basada en la integración de dos metodologías de verificación y simplificación de lógica: la homogeneización matemática de flujos de ejecución mediante el **Operador Binario Universal** y el razonamiento inductivo guiado mediante la técnica **Draft, Sketch, and Prove (Vibe Proving)**.

La solución no pretende sustituir los motores de Inteligencia Artificial base del mercado, sino orquestar APIs de LLMs de alto rendimiento (como Gemini Pro de Google) integrándolas en un pipeline cerrado y auto-verificado con asistentes de prueba matemáticos formales (**Lean 4**).

3.1. Arquitectura del Sistema

El proceso de auditoría y certificación consta de tres pasos modulares descritos en la Figura 1:

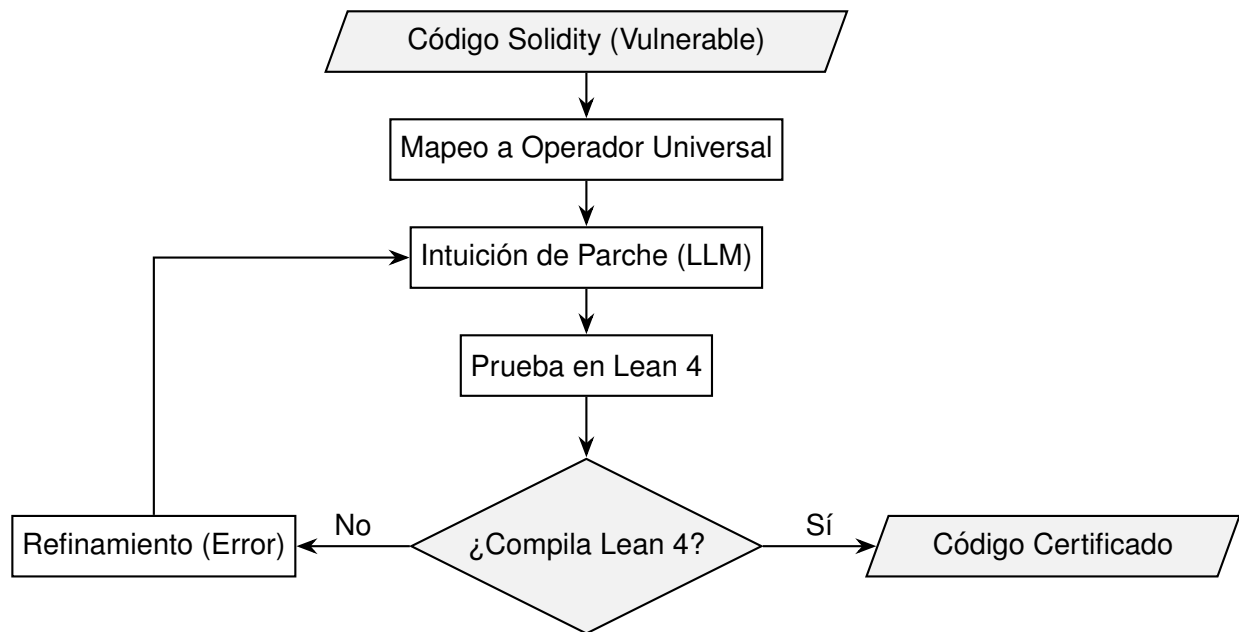


Figura 1: Diagrama de Flujo del Pipeline de Solidify-empl.

1. **Capa de Traducción (AST a Operador Universal):** El código Solidity del Smart Contract se parsea para extraer su Árbol de Sintaxis Abstracta (AST). Las invariantes y restricciones aritméticas clave del contrato (como el control de balances y las transferencias financieras de fondos) se traducen y homogeneizan mediante el operador universal de Odrzywolek. Por su parte, la lógica del flujo de control y las llamadas externas se modelan de manera aislada utilizando grafos de transición de estados tradicionales para evitar la distorsión del dominio continuo del operador.
2. **Capa de Intuición (Vibe Proving):** Una API de LLM (Gemini Pro) actúa como un intuicionista rápido. Analiza la representación simplificada de árbol del contrato y el exploit de reentrada de prueba. El modelo detecta el error de diseño lógico y genera dos entregables preliminares:
 - Un código parcheado sugerido.
 - Un borrador informal de la demostración matemática de seguridad que demuestra que, tras aplicar el parche, el saldo no puede descender a valores inválidos ni volver a invocarse la función de forma recursiva antes del cierre del estado. Este borrador informal incluye comentarios lógicos en línea que guiarán la generación del código Lean 4.
3. **Capa de Verificación (Lean 4 Loop):** El borrador de la prueba se traduce a la sintaxis del lenguaje de programación lógico Lean 4. Un compilador local de Lean 4 ejecuta la verificación formal. Si la prueba es matemáticamente válida, Lean 4 compila con éxito y el código corregido se declara inmune frente a reentrada bajo las especificaciones demostradas. Si falla, el mensaje de error del compilador se inyecta de vuelta en el LLM mediante un bucle de corrección cerrada (*Self-Repair Loop*) para que refine la prueba, repitiendo el ciclo de forma totalmente autónoma.

3.2. Estado de Desarrollo Actual

Actualmente, el proyecto se encuentra en fase de validación teórica, investigación y diseño de arquitectura preliminar. El desarrollo del código fuente correspondiente a las fases técnicas descritas se iniciará e implementará íntegramente durante la estancia de investigación en el centro GSEC Málaga.

4. Fundamentos Tecnológicos y Científicos

La innovación de **Solidify-eml** reside en la intersección de la teoría de operadores elementales y las técnicas modernas de guiado informal de demostradores de teoremas asistidos por Inteligencia Artificial.

4.1. El Operador Binario Universal de Odrzywólek

En matemáticas puras y teoría de la computación, es habitual buscar primitivas mínimas universales. Así como en la lógica digital booleana la puerta lógica NAND es un operador universal (capaz de implementar cualquier otra función lógica), en el análisis matemático clásico, el físico Andrzej Odrzywólek demostró en 2026 que el operador binario $\text{eml}(x, y)$ definido como:

$$f(x, y) = e^x - \ln(y) \quad (1)$$

es suficiente para generar todas las funciones matemáticas elementales (aritmética básica, funciones trigonométricas, logaritmos, potencias y raíces) cuando se aplica de manera recursiva sobre el número constante 1.

Por ejemplo, operaciones fundamentales se expresan de la siguiente forma a través de esta única primitiva:

- La función exponencial simple se deriva de forma directa: $e^x = f(x, 1) = e^x - \ln(1) = e^x$.
- La operación de resta se puede derivar mediante la combinación de logaritmos y exponenciales recursivas.
- La multiplicación y la división lógicas se unifican bajo esta misma primitiva a través del mapeo logarítmico.

4.1.1. Aplicación a la Reducción del Espacio de Estados

En la verificación formal clásica de software, cada instrucción diferente del lenguaje (ej. asignaciones de memoria, comparaciones, saltos JUMP, llamadas CALL) requiere un modelo formal con axiomas matemáticos propios, lo que incrementa el tamaño de la matriz de fórmulas lógicas exponencialmente.

En **Solidify-eml**, reducimos la representación de las invariantes lógicas y aritméticas financieras críticas del contrato inteligente (como los balances y la asignación de fondos de las cuentas del contrato) a un árbol binario homogéneo compuesto únicamente de nodos del operador de Odrzywólek. Al unificar el substrato de representación matemática de la lógica del programa:

1. Se elimina la complejidad accidental de tratar con múltiples instrucciones aritméticas independientes en el resolutor lógico.
2. El espacio de búsqueda del verificador formal se simplifica, reduciéndose a evaluar relaciones de equivalencia entre árboles binarios con un único tipo de nodo de cálculo.

4.1.2. Mitigación del Dominio y Singularidad de Lean 4

En el asistente de pruebas formales Lean 4, todas las funciones matemáticas deben ser declaradas como funciones totales. Por defecto, en la librería estándar de Lean 4 (`mathlib`), el logaritmo de cero se asigna a un valor basura arbitrario (específicamente, $\ln(0) = 0$). Esto puede dar lugar a inconsistencias lógicas si la traducción del árbol de Odrzywólek no restringe las variables.

Para solventar esta limitación en el compilador de **Solidify-eml**, el sistema implementa un pre-validador lógico que inyecta de forma obligatoria la precondition $y > 0$ para cada nodo evaluador del operador universal $\text{eml}(x, y)$. Al restringir formalmente el dominio a balances estrictamente positivos

en el flujo financiero del contrato inteligente, se garantiza la consistencia del teorema y se evita la singularidad matemática durante la compilación en Lean 4.

4.2. Draft, Sketch, and Prove (Vibe Proving)

La demostración formal deductiva es un problema complejo con un espacio de búsqueda inmenso. A diferencia de la fuerza bruta, el razonamiento matemático humano emplea heurísticas o intuición estructural para conjeturar un resultado y, posteriormente, formalizar los pasos lógicos intermedios necesarios para su demostración.

El framework *Draft, Sketch, and Prove* (DSP), desarrollado por Google DeepMind, formaliza este comportamiento en sistemas de inteligencia artificial:

1. **Borrador (Draft):** Un LLM escribe una demostración informal en lenguaje natural estructurado a partir del código del Smart Contract, prediciendo qué invariante de bucle o propiedad de estado (ej. $\text{balance} \geq 0$) evitará la vulnerabilidad.
2. **Bosquejo (Sketch):** El borrador informal se descompone en un “boceto” formal de alto nivel de la prueba en Lean 4. Este boceto define los hitos intermedios y subgoles lógicos de la demostración matemática. Para evitar la degradación del éxito del modelo (estimada entre el 2.8% y el 4.9% en la literatura experimental), el generador inyecta los comentarios informales de la fase de borrador como comentarios en línea directamente dentro de la estructura táctica de Lean 4.
3. **Prueba (Prove):** El demostrador Lean 4 intenta rellenar de forma mecánica y autónoma las brechas lógicas entre los hitos del boceto. Para emular la potente automatización declarativa del software Sledgehammer de Isabelle (usado en el paper original de DSP), Solidify-eml utiliza la integración del framework táctico extensible **aesop** (Automated Extensible Search Over Proofs) en Lean 4, junto con resolvers SMT externos configurados en el compilador.

5. Capacidades y Experiencia del Solicitante

El solicitante del proyecto, Alejandro Moyano Crespillo, se encuentra matriculado en la titulación oficial de Grado en Ingeniería en Sistemas Electrónicos de Telecomunicación en la Universidad de Málaga (UMA). La base de matemáticas aplicadas y arquitectura de sistemas adquirida durante esta formación académica de ingeniería proporciona las competencias necesarias para abordar la simplificación sintáctica de código (AST) y la orquestación del demostrador formal Lean 4 requeridos en este proyecto.

Sus competencias y áreas de estudio específicas incluyen:

- **Electrónica y Programación de Bajo Nivel:** Formación en el diseño de sistemas de procesamiento de datos, microcontroladores y lógica digital. Esto facilita la comprensión de las restricciones físicas y la simplificación binaria de instrucciones a bajo nivel (esencial para la optimización y traducción de AST de Solidity a operadores mínimos).
- **Arquitectura de Sistemas y Redes:** Formación en teoría de redes, protocolos de comunicaciones y fundamentos de seguridad en sistemas distribuidos, aplicable al análisis y flujo de datos de Smart Contracts.
- **Fundamentos Matemáticos e IA:** Capacidad de análisis y asimilación de modelos matemáticos continuos y teoría de control, clave para el modelado de la convergencia de la Inteligencia Artificial generativa y la verificación formal de teoremas en Lean 4.

El solicitante contará además con acceso a mentorías de profesionales del ecosistema de ciberseguridad en Málaga y recursos de la Cátedra de Ciberseguridad UMA-VirusTotal para complementar el desarrollo del proyecto.

Nota: De acuerdo con las bases de la convocatoria, se adjuntan el Currículum Vitae breve y la correspondiente Declaración Responsable de originalidad y aceptación del solicitante junto a la documentación de solicitud.

6. Plan de Trabajo y Cronograma

El proyecto se desarrollará a lo largo de un periodo intensivo de ocho semanas durante los meses de **Julio y Agosto de 2026**. El plan de trabajo está estructurado en cuatro fases quincenales consecutivas, garantizando un avance modular y la entrega de un Producto Mínimo Viable (MVP) funcional y demostrable al final del periodo en GSEC Málaga.

6.1. Fase 1 (Semanas 1-2): Compilador y Parser de Operador Universal

- **Objetivo:** Construir la capa de traducción del código fuente de Solidity a la representación binaria unificada.
- **Tareas:**
 - Implementar un parser en Python utilizando el compilador oficial de Solidity (`solc`) para extraer el AST (Abstract Syntax Tree) de funciones críticas.
 - Desarrollar el algoritmo de mapeo matemático para traducir las variables de estado financiero y las llamadas de control a expresiones binarias basadas exclusivamente en el operador universal $e^x - \ln(y)$ de Odrzywołek, acotando el compilador a un subconjunto restringido y específico de Solidity (operaciones básicas de balance, depósito y transferencia).
- **Entregable:** Módulo traductor Solidity-to-Universal-Operator funcional con casos de prueba de contratos estándar (ej. tokens ERC-20).

6.2. Fase 2 (Semanas 3-4): Pipeline de Intuición Artificial (Vibe Proving)

- **Objetivo:** Integrar los LLMs de Google mediante APIs para actuar como generadores rápidos de bocetos de prueba de seguridad.
- **Tareas:**
 - Diseñar las plantillas de prompts de ingeniería del sistema e inyectar el grafo simplificado de Operador Universal al LLM (Gemini Pro/Gemini 1.5).
 - Desarrollar el pipeline de generación del borrador informal de la demostración y el correspondiente parche contra vulnerabilidades de reentrada (*Reentrancy*), inyectando comentarios informales en el flujo de traducción.
- **Entregable:** Pipeline que recibe un contrato vulnerable en formato simplificado de operador universal y devuelve el borrador informal de seguridad y la propuesta de corrección en menos de 60 segundos.

6.3. Fase 3 (Semanas 5-6): Integración del Verificador Formal Lean 4

- **Objetivo:** Automatizar la compilación de pruebas formales y el bucle cerrado de autorreparación de código Lean 4.
- **Tareas:**
 - Implementar la traducción automática del boceto de prueba informal generado por el LLM a sintaxis ejecutable de Lean 4.

- Programar el bucle cerrado de retroalimentación (*Self-Repair Loop*): si Lean 4 devuelve fallos en el compilador, se inyectan las trazas de error al LLM de forma automática para la corrección iterativa de la prueba matemática.
- **Entregable:** Bucle autónomo de generación y verificación formal de teoremas integrado con Lean 4.

6.4. Fase 4 (Semanas 7-8): Evaluación de Resultados, Optimización y Demostración

- **Objetivo:** Validar la efectividad del framework sobre contratos reales vulnerables y preparar la demo final.
- **Tareas:**
 - Evaluar el rendimiento del MVP sobre un banco de pruebas de contratos vulnerables conocidos de la red Ethereum (como el contrato del exploit The DAO).
 - Medir y documentar la reducción del espacio de estados y la velocidad de verificación en comparación con métodos formales clásicos sin simplificación de operador.
 - Redacción de la memoria final del proyecto y preparación de la demostración interactiva en las oficinas de GSEC Málaga.
- **Entregable:** Memoria final de resultados del proyecto y framework de auditoría Solidify-eml operativo para la demostración del MVP.

6.5. Cronograma Resumido

A continuación, se detalla el cronograma de ejecución temporal del proyecto:

Fase	Actividad / Entregable	S1	S2	S3	S4	S5	S6	S7	S8
Fase 1	Parser y traductor Solidity → Operador Universal	•	•						
Fase 2	Integración API Google y Vibe Proving (Drafts)			•	•				
Fase 3	Pipeline de validación Lean 4 y Self-Repair Loop					•	•		
Fase 4	Pruebas empíricas, optimización y Demo GSEC							•	•

Tabla 1: Cronograma de Gantt del proyecto Solidify-eml (Julio-Agosto 2026).

7. Recursos Necesarios y Resultado Mínimo Esperado

7.1. Recursos de Computación e Infraestructura

Para la correcta ejecución del proyecto en los plazos establecidos, se requiere el uso de la infraestructura tecnológica proporcionada por Google en GSEC Málaga, detallada a continuación:

- **Recursos de Inteligencia Artificial (APIs):** Acceso prioritario y cuotas de API ampliadas a los modelos de lenguaje masivos de Google (**Gemini Pro** y **Gemini 1.5**) a través de Google Cloud Vertex AI para el análisis heurístico y la fase de generación del borrador informal de las pruebas de seguridad (*Vibe Proving*).
- **Infraestructura de Computación local/Cloud:** Un entorno de desarrollo local o instancia virtual en Google Cloud Platform (GCP) con al menos 16 vCPUs y 64 GB de RAM para ejecutar de manera paralela las múltiples instancias de compilación y verificación matemática rígida en Lean 4 durante las pruebas empíricas.

- **Mentoría Técnica:** Acceso a ingenieros y expertos de Google y VirusTotal ubicados en el centro de Málaga especializados en análisis estático de código, compiladores y metodologías de detección de amenazas de seguridad para evaluar y refinar el pipeline de Solidify-eml.

La dotación económica de los premios de la convocatoria facilitará la dedicación exclusiva del solicitante para su desarrollo presencial en el centro GSEC Málaga, permitiendo financiar los recursos de computación en la nube (APIs Vertex AI de Google Cloud) y los gastos de manutención y desplazamiento asociados a la estancia de dos meses.

7.2. Resultado Mínimo Esperado (MVP)

El producto mínimo esperado al finalizar la estancia de dos meses es una herramienta funcional de consola (CLI) escrita en Python que implemente el pipeline de Solidify-eml completo de principio a fin, certificado mecánicamente.

El MVP estará validado mediante la detección exitosa y la autocuración certificada formalmente (sin falsos positivos ni falsos negativos) de la vulnerabilidad de **reentrada (Reentrancy)** en un conjunto de al menos 10 contratos inteligentes de prueba reales recopilados de la red Ethereum (incluyendo código clásico vulnerable y sus correspondientes correcciones manuales y automáticas). El sistema debe ser capaz de:

1. Identificar la presencia del bug de reentrada en el contrato Solidity original.
2. Generar la corrección del código de forma automatizada.
3. Compilar un archivo de prueba matemática formal de seguridad en Lean 4 que demuestre de manera concluyente y verificable al 100 % que el contrato parcheado es inmune a ataques de reentrada.

7.3. Escalabilidad y Líneas de Trabajo Futuro

Una vez validada la viabilidad del MVP durante la estancia del concurso en Málaga, la tecnología del Operador Binario Universal unida al guiado informal de demostradores mediante IA abre oportunidades de negocio altamente escalables a largo plazo:

- **Ampliación del Catálogo de Vulnerabilidades:** Extender el compilador de operador universal y el sistema de pruebas formales a otras vulnerabilidades críticas de DeFi (como fallos en el control de acceso, desbordamientos de enteros o manipulación de oráculos de precios).
- **Integración en pipelines de CI/CD:** Integrar la herramienta de certificación como un paso de validación obligatorio y automatizado en plataformas de desarrollo como GitHub Actions antes del despliegue de contratos en redes principales (*Mainnet*).
- **Servicio de Auditoría Automatizada (SaaS):** Crear una plataforma web donde desarrolladores de blockchain y empresas de DeFi puedan subir sus contratos inteligentes y obtener un certificado formal matemático de seguridad 100 % verificado en segundos a una fracción del coste de una auditoría manual.

Referencias

- [1] A. Odrzywolek, “All elementary functions from a single binary operator”, *arXiv preprint arXiv:2603.21852*, 2026. Disponible en: <https://arxiv.org/abs/2603.21852> (última consulta: 5 de julio de 2026).
- [2] A. Q. Jiang *et al.*, “Draft, Sketch, and Prove: Guiding Formal Theorem Provers with Informal Proofs”, *arXiv preprint arXiv:2210.12283*, *ICLR 2023*. Disponible en: <https://arxiv.org/abs/2210.12283> (última consulta: 5 de julio de 2026).
- [3] L. de Moura y S. Ullrich, “The Lean 4 Theorem Prover and Programming Language”, en *International Conference on Automated Deduction (CADE 2021)*, 2021. Disponible en: <https://arxiv.org/abs/2108.06730> (última consulta: 5 de julio de 2026).